

Vectores

La situación problemática

Un enunciado típico que con lo que sabemos hoy se vuelve incómodo de resolver.

Dadas las edades de los 20 alumnos de un curso, informar cuántos de ellos **tienen una edad mayor a la edad promedio**.

¿Cómo lo resolveríamos hoy?

Opción 1 · Una variable distinta para cada alumno

```
int edad1, edad2, edad3, edad4, edad5,  
    edad6, edad7, edad8, edad9, edad10,  
    edad11, edad12, /* ... */  
    edad19, edad20;  
  
cin >> edad1 >> edad2 >> /* ... */ >> edad20;  
  
int promedio = (edad1 + edad2 +  
    /* ... 17 sumandos más ... */  
    + edad20) / 20;
```

Se vuelve inmanejable

20 nombres distintos para pensar y escribir a mano.

Para comparar cada alumno con el promedio hay que repetir el mismo "if" 20 veces, uno por variable.

¿Y si en vez de 20 alumnos son 200? El código no escala: hay que reescribirlo entero.

¿Cómo lo resolveríamos hoy?

Opción 2 · Cargar con un for, calcular el promedio... ¿y después?

```
int suma = 0, edad;

for (int i = 0; i < 20; i++) {
    cin >> edad;
    suma = suma + edad;
}

float promedio = suma / 20.0;
```

El problema: ¿dónde quedó cada edad?

La variable "edad" se pisa en cada vuelta del for: al terminar el ciclo solo queda la última que se cargó.

Ya tenemos el promedio... pero para saber cuántos alumnos superan ese promedio habría que volver a pedir las 20 edades por teclado y compararlas una por una.

Pedirle al usuario los mismos datos dos veces no es una solución razonable.

La solución: un vector

Muchos datos del mismo tipo, bajo un solo nombre, uno al lado del otro en memoria.

Un vector es una estructura que guarda una cantidad fija de elementos del mismo tipo, accesibles todos con el mismo nombre y un número de posición (índice).

edades	18	22	16	...	21
índice	0	1	2	3	4

Un nombre, muchas casillas

"edades" es un solo nombre que representa las 20 casillas.

Cada casilla guarda un valor y se identifica por su posición.

Tamaño fijo

La cantidad de elementos se define al declararlo y no cambia.

En C++ un vector de tamaño fijo se declara como un arreglo (array): tipo nombre[cantidad];

Declaración de un vector

Tipo de los elementos, nombre y, entre corchetes, la cantidad fija de elementos.

Sintaxis general

```
int vec[10];    // forma general

int edades[20];
float notas[10];
```

Se lee de a partes

```
int edades[20];
```

tipo de cada elemento · nombre del vector · cantidad de elementos

Otras formas de declararlo

```
int edades[5];           // crea 5 casillas de tipo int, sin valor definido
int edades[5] = {18, 22, 16, 19, 21}; // con valores iniciales

const int CANT_ALUMNOS = 20;
int edades[CANT_ALUMNOS]; // el tamaño también puede venir de una constante
```

El tamaño se fija en la declaración: no se puede agrandar ni achicar durante la ejecución del programa.

Acceder a las posiciones

Cada elemento se accede con nombre[índice]. El primer elemento está en la posición 0.

edades	18	22	16	19	21
	0	1	2	3	4

`edades[2]` contiene 16

En código

```
cout << edades[0]; // 18
cout << edades[2]; // 16

edades[4] = 30; // modifica
                // la última
posición

edades[5]; // ¡error!
           // no existe la
posición 5
```

Un vector de 5 elementos usa los índices 0, 1, 2, 3 y 4. El índice 5 está fuera de rango.

El problema, resuelto

Ahora sí: cargar, calcular el promedio y comparar, sin perder ningún dato.

```
const int CANT = 20;
int edades[CANT];
int suma = 0;

// 1) cargar y acumular la suma
for (int i = 0; i < CANT; i++) {
    cin >> edades[i];
    suma = suma + edades[i];
}

// 2) calcular el promedio
float promedio = suma / (float)CANT;

// 3) comparar cada edad contra el promedio
int cant_mayores = 0;
for (int i = 0; i < CANT; i++) {
    if (edades[i] > promedio)
        cant_mayores++;
}
```

¿Qué cambió?

El vector guarda las 20 edades: el primer for las carga y las conserva, el segundo for las vuelve a recorrer y las compara contra el promedio. Ningún dato se vuelve a pedir.

El patrón se repite siempre

1. Declarar el vector con su tamaño fijo.
2. Recorrerlo con un for para cargarlo.
3. Recorrerlo (las veces que haga falta) para calcular, buscar o comparar.

El vector es lo que permite separar "cargar" de "usar" los datos.

Y si necesito dos dimensiones: la matriz

Una tabla de filas y columnas: un vector de tamaño fijo, pero en dos dimensiones.

Una matriz organiza los datos en filas y columnas. Cada elemento se ubica con DOS índices: uno para la fila y otro para la columna.

		columna →			
		0	1	2	3
fila	0	7	8	6	9
	1	5	10	7	8
	2	9	6	8	7

Ejemplo: notas por alumno

3 filas (alumnos) × 4 columnas (parciales): `notas[1][2]` apunta a la nota del alumno 1 en el parcial 2 → 7.

Tamaño fijo, en dos dimensiones

La cantidad de filas y de columnas se define al declarar la matriz y no cambia durante la ejecución, igual que en un vector de una sola dimensión.

Matrices en C++

Un arreglo de dos dimensiones: se declara y se accede con dos pares de [].

Sintaxis general

```
int mat[10][20];  
  
// 10 filas, 20 columnas
```

Ejemplo aplicado (3 filas × 4 columnas)

```
const int FILAS = 3, COLS = 4;  
int notas[FILAS][COLS];  
  
// ejemplo aplicado: 3 alumnos, 4 notas c/u
```

Acceso: nombre[filas][columna]

```
notas[1][2] = 7;           // fila 1, columna 2 → alumno 1, parcial 2  
cout << notas[1][2];      // muestra 7  
  
// recorrer TODA la matriz siempre necesita dos for anidados  
for (int f = 0; f < FILAS; f++) {  
    for (int c = 0; c < COLS; c++) {  
        cin >> notas[f][c]; // f: fila actual, c: columna actual  
    }  
}
```

Igual que en el vector, los índices de fila y de columna arrancan en 0.

¿Preguntas?

Vector: nombre[índice] · Matriz: nombre[filas][columnas] · Índices siempre desde 0